Table des matières

REMERCIEMENTS....2
INTRODUCTION GÉNÉRALE....3
CHAPITRE I : CADRE THÉORIQUE SUR LES SGBD ET LA RÉPLICATION....4
1.1. Les Systèmes de Gestion de Bases de Données (SGBD)....4
1.2. Le Concept de Réplication....4
CHAPITRE II : DÉPLOIEMENT ET CONFIGURATION DU SERVEUR MAÎTRE....5
2.1. Mise à jour du système et Installation....5
2.2. Configuration du Pare-feu (UFW)....5
2.3. Paramétrage du fichier de configuration MariaDB....5
2.4. Création de l'utilisateur de réplication....6
CHAPITRE III : DÉPLOIEMENT ET CONFIGURATION DU SERVEUR ESCLAVE....8
3.1. Installation et préparation....8
3.2. Connexion de l'Esclave au Maître....8
3.3. Vérification de la synchronisation....8
CHAPITRE IV : PROBLÉMATIQUES RENCONTRÉES ET SOLUTIONS APPORTÉES....10
4.1. Conflits de Pare-feu (Firewall)....10
4.2. Ambition Multi-Esclaves et Synchronisation....10
CONCLUSION GÉNÉRALE....11
BIBLIOGRAPHIE ET WEBOGRAPHIE....12

REMERCIEMENTS

La réalisation de ce travail pratique n'aurait pu aboutir sans la contribution et le soutien de plusieurs personnes. Nous tenons tout d'abord à exprimer notre profonde gratitude au Professeur KASENGEDIA MOTUMBE Pierre pour ses enseignements de qualité dans le cadre du cours de Système d'Exploitation, ainsi qu'au Doctorant Junior et à l'Assistant Ferdinand pour leur accompagnement et leurs orientations techniques.

Nous remercions également tous les membres du Groupe 23 pour leur esprit d'équipe, leur rigueur dans la recherche des solutions face aux bugs rencontrés sous notre environnement Linux, et leur dévouement tout au long de ce projet.

INTRODUCTION GÉNÉRALE

Dans le cadre de l'enseignement des Systèmes d'Exploitation, la maîtrise théorique doit impérativement s'accompagner d'une mise en pratique rigoureuse. Ce travail pratique a été conçu pour valider l'acquisition des compétences liées à l'administration de serveurs sous un environnement Linux. Aujourd'hui, les données constituent le cœur de toute organisation, et leur disponibilité constante est une exigence absolue.

Le déploiement d'un serveur de base de données revient à configurer un système de gestion de base de données pour qu'il soit opérationnel sur un ordinateur. Cependant, un serveur isolé représente un point de défaillance unique (Single Point of Failure). C'est pourquoi nous abordons ici le déploiement d'un SGBD MariaDB couplé à l'implémentation d'une stratégie de réplication. L'objectif principal de cette démarche est de prévenir la perte de données et d'assurer une haute disponibilité du système en cas de panne matérielle ou logicielle.

La réplication consiste à utiliser une ou plusieurs autres machines (serveurs auxiliaires ou de secours) pour sauvegarder et synchroniser les données du serveur principal, tout en configurant les autorisations d'administration adéquates. Nous avons suivi toutes les étapes enseignées durant le cours pour installer, initialiser et configurer notre serveur avec MySQL via son extension MariaDB.

Ce document illustre les différentes étapes qui nous ont conduits à la réalisation de notre projet académique, ainsi que les différents tests effectués sur nos serveurs. Parmi ces tests, nous avons effectué des ajouts et des suppressions d'éléments dans une base de données pour nous assurer du bon fonctionnement de la réplication. Étant conscients qu'aucune œuvre humaine n'est parfaite, nous prions le lecteur de faire preuve d'indulgence quant aux éventuelles imperfections de ce rapport.

CHAPITRE I : CADRE THÉORIQUE SUR LES SGBD ET LA RÉPLICATION

1.1. Les Systèmes de Gestion de Bases de Données (SGBD)

Un SGBD est un logiciel système destiné à stocker, manipuler et gérer des bases de données. Dans le cadre de ce projet, notre choix s'est porté sur MySQL et son fork *open-source*, MariaDB. MariaDB offre l'avantage d'être nativement intégré dans les dépôts de la plupart des distributions Linux, garantissant une grande stabilité, une exécution rapide et une sécurité renforcée.

1.2. Le Concept de Réplication

La réplication de base de données est un processus automatisé de copie et de maintien des données d'une base de données (le Maître) vers une ou plusieurs autres bases de données (les Esclaves). Cette architecture présente plusieurs avantages majeurs pour l'administration système :

- **Sécurité des données (Backups) :** L'esclave contient une copie exacte des données du maître.
- **Haute Disponibilité :** Si le serveur principal tombe en panne, le serveur de secours peut prendre le relais, minimisant ainsi le temps d'arrêt.
- **Répartition de la charge (Load Balancing) :** Les requêtes de lecture peuvent être dirigées vers l'esclave, déchargeant ainsi le maître qui se concentre sur les requêtes d'écriture.

Dans notre infrastructure, nous avons opté pour une topologie "Maître-Esclave" (Master-Slave), où les écritures se font exclusivement sur le serveur principal, et les modifications sont répliquées de manière asynchrone ou semi-synchrone sur les serveurs de secours.

CHAPITRE II : DÉPLOIEMENT ET CONFIGURATION DU SERVEUR MAÎTRE

Le déploiement du serveur principal (Maître) nécessite une configuration minutieuse, car il dictera le comportement de l'ensemble de l'architecture de réplication. Nous avons effectué cette installation depuis le terminal Ubuntu.

2.1. Mise à jour du système et Installation

Avant toute installation logicielle sous Linux, il est impératif de mettre à jour la liste des paquets. Après l'ouverture du terminal Bash , nous avons rafraîchi l'environnement et procédé à l'installation:

```
sudo apt update && sudo apt upgrade sudo apt install
mariadb-server -y
```

Le paramètre `-y` permet de valider automatiquement les invites d'installation. Nous avons ensuite vérifié la bonne installation avec la commande `mariadb --version`.

2.2. Configuration du Pare-feu (UFW)

La sécurité réseau est cruciale. Le serveur de base de données écoute généralement sur le port 3306. Il faut s'assurer que le pare-feu (Uncomplicated Firewall) permet le trafic tout en protégeant le système. Nous l'avons activé via la commande :

```
sudo ufw enable
```

(Note : Il est également nécessaire d'autoriser spécifiquement le trafic MySQL avec `sudo ufw allow 3306` ou `sudo ufw allow mysql` pour que l'esclave puisse se connecter).

2.3. Paramétrage du fichier de configuration MariaDB

Par défaut, MariaDB est configuré pour n'écouter que sur l'adresse locale (`127.0.0.1`), ce qui empêche toute connexion externe. Il a fallu modifier les paramètres pour rendre le PC maître visible par les PC qui stockeront les serveurs de secours. Nous avons édité le fichier de configuration avec l'éditeur de texte en ligne de commande `nano` :

```
sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

Dans ce fichier, les modifications clés incluent :

1. **Changement de l'adresse d'écoute :** `bind-address = 0.0.0.0` (pour écouter sur toutes les interfaces réseau).
2. **Activation du journal binaire :** Dé-commenter `log_bin = /var/log/mysql/mysql-bin.log` (indispensable pour que le maître enregistre les changements à envoyer à l'esclave).
3. **Attribution de l'ID serveur :** Définir `server-id = 1` (Chaque machine dans la réplication doit avoir un ID unique).

Nous avons ensuite redémarré le service pour appliquer les modifications apportées au fichier:

```
sudo systemctl restart mariadb
```

2.4. Création de l'utilisateur de réplication

L'esclave a besoin de privilèges spécifiques pour récupérer les données du maître. Après avoir lancé l'invite de commande MariaDB via `sudo mariadb` , nous avons configuré les paramètres pour que le maître puisse répliquer sur un autre serveur:

```
CREATE USER 'replique'@'%' IDENTIFIED BY 'Password'; GRANT
REPLICATION SLAVE ON *.* TO 'replique'@'%'; FLUSH
PRIVILEGES;
```

Enfin, il faut figer l'état du maître pour obtenir les coordonnées exactes du fichier journal (log) à partir duquel l'esclave doit commencer à copier :

```
SHOW MASTER STATUS;
```

Cette commande nous a permis d'obtenir le nom du fichier (`File`) et la position (`Position`) que les serveurs de secours doivent utiliser. (Exemple : fichier `mariadb-bin.000001` et position `330`) .

```
MariaDB [(none)]> SHOW MASTER STATUS;
+--------------------+----------+--------------+------------------+
| File               | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+--------------------+----------+--------------+------------------+
| mariadb-bin.000022 |     6426 |              |                  |
+--------------------+----------+--------------+------------------+
1 row in set (0,000 sec)
```

CHAPITRE III : DÉPLOIEMENT ET CONFIGURATION DU SERVEUR ESCLAVE

Le serveur de secours (Esclave) a pour rôle de lire en permanence le journal binaire du maître et de rejouer les requêtes localement.

3.1. Installation et préparation

Les étapes initiales sont identiques à celles du maître : rafraîchissement du bash, installation du serveur MariaDB et vérification de la version.

Nous avons également modifié le fichier `50-server.cnf` via la commande `sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf`. L'élément critique ici est d'attribuer un identifiant unique (par exemple, `server-id = 2`). Un redémarrage du service a suivi : `sudo systemctl restart mariadb`.

3.2. Connexion de l'Esclave au Maître

Après le lancement de MariaDB, il faut informer l'esclave de l'identité du maître et lui fournir les identifiants de connexion. Nous avons utilisé les commandes de configuration du serveur suivantes:

```
STOP SLAVE; CHANGE MASTER TO MASTER_HOST='IP adress serveur maître', MASTER_USER='replique', MASTER_PASSWORD='le mot de passe définit par le maître', MASTER_LOG_FILE='le nom du fichier de connexion du maître', MASTER_LOG_POS=la position sur laquelle est connecter le maître; START SLAVE;
```

3.3. Vérification de la synchronisation

Pour confirmer que la connexion entre le serveur principal et le serveur de réplication a été créée avec succès, nous avons vérifié le statut :

```
SHOW SLAVE STATUS\G;
```

```
MariaDB [(none)]> SHOW SLAVE STATUS\G;
*************************** 1. row ***************************
               Slave_IO_State: Waiting for master to send event
                  Master_Host: 10.42.114.234
                  Master_User: Poste2
                  Master_Port: 3306
                Connect_Retry: 60
              Master_Log_File: mariadb-bin.000022
          Read_Master_Log_Pos: 6426
               Relay_Log_File: mysqld-relay-bin.000012
                Relay_Log_Pos: 557
        Relay_Master_Log_File: mariadb-bin.000022
             Slave_IO_Running: Yes
            Slave_SQL_Running: Yes
              Replicate_Do_DB:
          Replicate_Ignore_DB:
           Replicate_Do_Table:
       Replicate_Ignore_Table:
      Replicate_Wild_Do_Table:
  Replicate_Wild_Ignore_Table:
                   Last_Errno: 0
                   Last_Error:
                 Skip_Counter: 0
          Exec_Master_Log_Pos: 6426
              Relay_Log_Space: 867
              Until_Condition: None
               Until_Log_File:
                Until_Log_Pos: 0
           Master_SSL_Allowed: No
           Master_SSL_CA_File:
```

Nous avons également configuré le délai de tentative de reconnexion pour avoir une synchronisation toutes les deux secondes sur le serveur de secours:

```
STOP SLAVE; SET GLOBAL MASTER_CONNECT_RETRY = 2; START SLAVE;
```

Afin que le service démarre automatiquement au lancement du système, nous avons activé le démon : `sudo systemctl enable mariadb`.

CHAPITRE IV : PROBLÉMATIQUES RENCONTRÉES ET SOLUTIONS APPORTÉES

La mise en place d'une architecture distribuée n'est jamais sans embûches. Ce projet académique a été riche en enseignements grâce aux obstacles techniques que nous avons dû surmonter.

4.1. Conflits de Pare-feu (Firewall)

La plus grande difficulté rencontrée concernait la connexion du PC maître avec le PC esclave. Bien que la configuration réseau et les identifiants MariaDB fussent corrects, la communication était bloquée parce que le pare-feu du PC hébergeant le serveur maître était désactivé de manière inattendue ou mal configuré. Cela nous a poussés à approfondir nos recherches pour gérer finement l'utilitaire `ufw`. La solution a consisté à activer correctement `ufw` et à s'assurer que les règles entrantes autorisaient spécifiquement l'adresse IP de l'esclave sur le port 3306, garantissant ainsi une réplication avec succès et en toute sécurité.

4.2. Ambition Multi-Esclaves et Synchronisation

La réplication consiste à créer une copie de notre serveur sur un PC différent tout en restant synchronisé chaque fois pour permettre une mise à jour instantanée des données. Outre le défi de la connexion maître-esclave, nous avons été confrontés à un problème de synchronisation lorsque nous avons voulu ajouter des esclaves supplémentaires.

Cependant, nous avons continué à tenter une réplication sur deux esclaves. Notre ambition était de réaliser un test avec plusieurs esclaves et d'observer leur comportement avec tous les droits qui leur étaient assignés. Ces tests à grande échelle ont mis en évidence l'importance de s'assurer que les identifiants `server-id` soient rigoureusement uniques sur chaque nœud du réseau, et que le `MASTER_LOG_POS` soit lu avec précision au moment de la création du *dump* initial des données.

CONCLUSION GÉNÉRALE

Ce projet nous a permis de franchir un cap important : passer de la théorie abordée en cours à la pratique concrète. En utilisant le terminal Ubuntu, nous avons réussi à configurer de bout en bout un système d'exploitation distribué axé sur la gestion de bases de données.

Nous avons démontré qu'une réplication est non seulement fonctionnelle, mais aussi indispensable, comme l'ont prouvé nos tests d'intégrité (ajouts et suppressions de données avec observation de la synchronisation instantanée). De plus, ce travail a mis en lumière l'importance critique de la configuration du réseau et de la sécurité, notamment à travers la gestion rigoureuse du pare-feu.

Les difficultés rencontrées avec l'architecture multi-esclaves ouvrent de nouvelles perspectives d'apprentissage pour la suite de notre cursus universitaire. Nous ressortons de cette expérience avec des compétences techniques consolidées en administration de bases de données sous Linux, prêts à concevoir des architectures résilientes et hautement disponibles. Merci beaucoup.

BIBLIOGRAPHIE ET WEBOGRAPHIE

Support de cours :

1. KASENGEDIA MOTUMBE, Pierre. *Notes du cours de Système d'Exploitation.* Université de Kinshasa, Faculté des Sciences et Technologies, Année Académique 2025-2026.

Outils d'Intelligence Artificielle et de Recherche :

2. Google Gemini. *Assistance à la recherche de commandes d'administration Linux et MariaDB, et résolution des problèmes de pare-feu UFW*. Consulté en cours de réalisation du projet.

3. Documentation Officielle de MariaDB : *Setting Up Replication*. [En ligne]. Disponible sur : mariadb.com/kb/en/setting-up-replication/ 4. Documentation Ubuntu Server : *Database Servers - MySQL/MariaDB*. [En ligne]. Disponible sur : ubuntu.com/server/docs